

Descripción de la Arquitectura del Software (SAD)

Sistema Integral de Gestión de Eventos

Equipo Hexacore

Versión: 1.1

Fecha de entrega: 27/08/2026

Autores:

Daniel Cristancho

Samuel Emperador

Sebastián Sánchez

Diego Coronado

Materia: Arquitectura de Software

Historial de Cambios

Fecha	Descripción del cambio	Persona(s)
25/08/2026	Creación de la versión 1.0 del documento, consolidando el trabajo ya elaborado en <i>ArchitecturalProposal.tex</i> (atributos de calidad y propuesta arquitectónica), <i>C4Diagrams.tex</i> (los seis diagramas C4), <i>Submission/DistribucionCasosUso.pdf</i> (los 32 casos de uso) y <i>BitacoraArquitectura.md</i> (decisiones, alternativas y riesgos). Se añade además un primer Árbol de Utilidad (ASR) y los ADR de las decisiones fundacionales, que hasta ahora solo existían en la bitácora.	Equipo Hexacore (vía asistente)
27/08/2026	Versión 1.1: se cierran los puntos pendientes de la v1.0 — diagrama de clases UML del modelo de dominio, vista de componentes de los contenedores Personal, Eventos/Emergencias y Parqueaderos, ambientes de desarrollo y pruebas en la vista física, y modelo de datos detallado por microservicio.	Equipo Hexacore (vía asistente)

Tabla de Contenidos

Historial de Cambios	1
1 Introducción	3
2 Visión general de los requisitos funcionales	3
3 Modelo de dominio	4
4 Stakeholders e intereses	7
5 Requisitos Arquitectónicamente Significativos (ASR)	7
6 Restricciones	10
7 Contexto y Alcance	10
8 Vista de contenedores	12
9 Vista de componentes	13
9.1 Servicio de Personal	13
9.2 Servicio de Eventos / Emergencias	13
9.3 Servicio de Parqueaderos	14
10 Vista de procesos	14
10.1 Reventa de una entrada (CU-006)	14
10.2 Evacuación ante emergencias (CU-010)	15
11 Vista física	15
12 Modelo de datos	16
13 Registros de Decisiones Arquitectónicas (ADR)	17
14 Riesgos técnicos	19
15 Glosario	20
16 Referencias	21

1. Introducción

El **Sistema Integral de Gestión de Eventos** centraliza en una sola plataforma los procesos que hoy suelen estar dispersos en la organización de un evento masivo (conciertos, festivales, partidos deportivos): venta de entradas y su mercado secundario, gestión de personal y turnos, parqueaderos, pedidos y establecimientos de alimentos, logística y operación del evento, y la administración general (usuarios, roles, recintos, proveedores, pagos y reportes). El problema central que resuelve es la **ausencia de un sistema unificado** que coordine estos procesos de forma consistente, segura y escalable, evitando información duplicada, errores humanos y falta de trazabilidad, especialmente crítico cuando hay miles de asistentes concurrentes durante la apertura de venta o el ingreso al recinto.

Los principales objetivos de negocio del sistema son: (1) permitir la venta y reventa segura de entradas sin duplicar propietarios ni sobrevender localidades; (2) mantener la disponibilidad del servicio durante los picos de tráfico propios de la apertura de venta, el ingreso masivo y las emergencias; (3) dar trazabilidad y auditoría a las operaciones financieras y administrativas; y (4) ofrecer una experiencia adecuada a cada tipo de usuario (cliente, personal operativo, administración) sin que ninguna interfaz duplique lógica de negocio.

Los atributos de calidad de mayor prioridad para la arquitectura son **consistencia, disponibilidad, seguridad y rendimiento**, seguidos de **tiempo real, escalabilidad y trazabilidad**; con prioridad baja-media se consideran **usabilidad, mantenibilidad y portabilidad**. La sección 5 detalla estos atributos en formato de Árbol de Utilidad.

Este documento sigue la estructura estándar *arc42* y se organiza así: primero se describen los requisitos funcionales y el modelo de dominio (secciones 2-3), luego los interesados y los requisitos arquitectónicamente significativos (secciones 4-5), después las restricciones del proyecto (sección 6) y las vistas arquitectónicas C4 (contexto, contenedores, componentes, procesos y física; secciones 7-11), y finalmente el modelo de datos, los registros de decisión (ADR), los riesgos técnicos, el glosario y las referencias.

2. Visión general de los requisitos funcionales

El sistema define **32 casos de uso** (CU-001 a CU-032), cada uno con un identificador único usado a lo largo de todo el documento. La especificación detallada de cada uno (actores, flujo principal, flujos alternativos y excepciones) está en la hoja de cálculo *Submission/CU_eventos_completo.xlsx*; la Tabla 2 resume los bloques temáticos en los que se agrupan.

Bloque temático	Casos de uso	Alcance funcional
Boletería / Compra de entradas	CU-001 – CU-005	Consulta de eventos, compra, validación de QR en el ingreso, cancelaciones/devoluciones y promociones.

Bloque temático	Casos de uso	Alcance funcional
Mercado secundario y personal para eventos	CU-006 – CU-010	Reventa segura de entradas (caso complejo), administración y asignación de personal, cambios de turno, registro de asistencia y gestión de evacuación ante emergencias (caso complejo).
Pedidos y establecimientos de alimentos	CU-011 – CU-015	Pedido de alimentos durante el evento, administración de menú/inventario, preparación y entrega validada por QR, y gestión de los establecimientos.
Logística y operación del evento	CU-016 – CU-020	Planificación logística, asignación de personal operativo, gestión de turno/asistencia en campo, monitoreo del evento y gestión de incidentes.
Parqueaderos	CU-021 – CU-025	Reserva, ingreso/salida de vehículos, asignación de espacios, cobro y control de ocupación.
Administración general (CRUD)	CU-026 – CU-032	Gestión de eventos, cuentas de usuario, roles y permisos, recintos y zonas, proveedores, pagos/conciliación financiera, y reportes y analítica.

Tabla 2: Bloques temáticos de los 32 casos de uso del sistema.

Los casos de uso están repartidos entre los cuatro integrantes del equipo (8 por persona); esa distribución, con nombre y descripción completa de cada CU, está documentada en *Submission/DistribucionCasosUso.pdf*. Dos casos de uso se destacan como los de mayor complejidad arquitectónica y sirven de hilo conductor en las vistas de este documento: **CU-006 (Gestión del Mercado Secundario de Entradas)** y **CU-010 (Gestionar Evacuación ante Emergencias)**.

3. Modelo de dominio

La Tabla 3 resume los conceptos del dominio identificados a partir de los 32 casos de uso. La Figura 1 presenta el diagrama de clases UML correspondiente (versión simplificada, con atributos clave y multiplicidades de las asociaciones principales); un modelo con navegabilidad, tipos de dato exactos y restricciones se refinará junto con el modelo entidad-relación de cada microservicio (sección 12) durante el prototipo.

Concepto	Explicación
Evento	Actividad (concierto, festival, partido) que un organizador crea, con fecha, recinto, localidades, precios y aforo.
Entrada	Boleto asociado a una localidad de un evento, con un código QR único que identifica al propietario actual.

Concepto	Explicación
Publicación de reventa	Oferta de una entrada ya adquirida, puesta en venta por su propietario en el mercado secundario.
Recinto / Zona	Lugar físico donde se realiza el evento y sus subdivisiones (zonas, localidades), cada una con capacidad y mapa de distribución.
Usuario / Cuenta	Persona registrada en el sistema (cliente, empleado, organizador o administrador), con credenciales y un conjunto de roles asignados.
Rol / Permiso	Conjunto de acciones autorizadas sobre un módulo del sistema, asignado a una cuenta de usuario.
Personal / Empleado	Persona que trabaja en la operación de un evento, con un rol operativo (seguridad, logística, alimentos, etc.).
Turno / Asignación	Periodo de trabajo de un empleado en un evento, con horario y zona asignada.
Registro de asistencia	Marca de ingreso y salida de un empleado durante un turno, usada para calcular horas trabajadas.
Parqueadero / Espacio	Zona de estacionamiento del evento y cada unidad de espacio reservable dentro de ella.
Pedido	Solicitud de productos de alimentos hecha por un cliente a un establecimiento durante un evento.
Establecimiento / Producto	Punto de venta de alimentos dentro del evento y los productos de su menú.
Pago	Transacción monetaria asociada a una compra, reventa, pedido o servicio de parqueadero, procesada mediante la pasarela de pagos externa.
Proveedor	Tercero externo (catering, sonido, seguridad privada, etc.) contratado para la operación de un evento.
Incidente / Protocolo de evacuación	Evento anómalo registrado durante la operación (desde uno menor hasta una emergencia que activa un protocolo de evacuación).
Reporte	Consulta agregada (ventas, ocupación, ingresos, personal, incidentes) usada para la toma de decisiones.

Tabla 3: Conceptos del dominio identificados a partir de los 32 casos de uso.

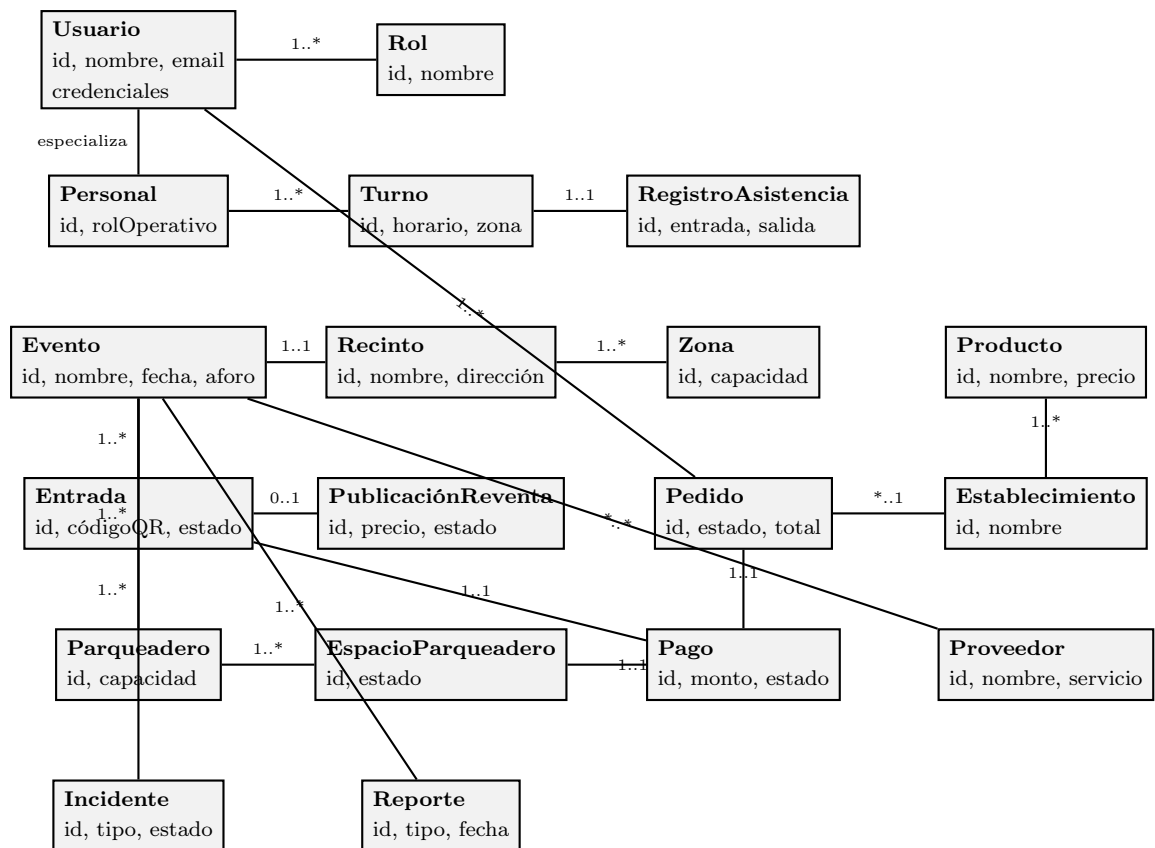


Figura 1: Diagrama de clases UML — modelo de dominio (versión simplificada).

4. Stakeholders e intereses

Rol/Nombre	Información de Contacto	Intereses / expectativas
Cliente	Rol dentro del sistema (se autentica con su cuenta)	Comprar y revender entradas de forma confiable, consultar sus pedidos y parqueadero, y recibir su QR sin fallas en el ingreso.
Organizador	Rol dentro del sistema	Crear y planificar eventos, asignar personal y proveedores, y monitorear la operación en tiempo real.
Empleado / Personal operativo	Rol dentro del sistema	Consultar sus turnos y asignaciones, solicitar cambios de turno y registrar su asistencia sin fricción.
Administrador	Rol dentro del sistema	Gestionar usuarios, roles, recintos y configuración general con control de acceso auditable.
Supervisor de emergencia	Rol dentro del sistema	Activar y coordinar protocolos de evacuación con información confiable y actualizada en tiempo real.
Proveedor externo	Gestionado por el organizador (módulo Proveedores)	Ser asignado correctamente a los eventos que contrata y recibir la información logística necesaria.
Equipo Hexacore (Daniel Cristancho, Samuel Emperador, Sebastián Sánchez, Diego Coronado)	<code>semperadorcontreras@gmail.com</code> (contacto del equipo)	Diseñar y sustentar una arquitectura que satisfaga los atributos de calidad priorizados y cumpla los requisitos del curso.
Profesor / Evaluador del curso	Curso de Arquitectura de Software	Verificar que el SAD cumpla la plantilla exigida y que las decisiones arquitectónicas estén justificadas.

5. Requisitos Arquitectónicamente Significativos (ASR)

La Tabla 5 presenta el primer Árbol de Utilidad del proyecto, derivado de los atributos de calidad priorizados en *ArchitecturalProposal.tex* y de los escenarios ya identificados en la bitácora arquitectónica.

ID ASR	Título	Atributo	Sub-atributo	CU(s)	Escenario	Prioridad
ASR-01	Bloqueo de doble venta en reventa	Consistencia	Concurrencia de datos	CU-006	Dos compradores intentan adquirir simultáneamente la misma entrada publicada en reventa; el sistema garantiza que solo uno complete la compra y el otro reciba de inmediato un mensaje de no disponibilidad.	Alta
ASR-02	Disponibilidad durante el ingreso masivo	Disponibilidad	Tolerancia a fallos	CU-002, CU-021, CU-022	Ante la caída de una instancia del servicio de validación de QR durante el ingreso al recinto, el balanceador redirige el tráfico a instancias sanas sin interrupción perceptible para el asistente.	Alta
ASR-03	Protección de datos de pago y credenciales	Seguridad	Confidencialidad	CU-001, CU-024, CU-031	Un intento de acceso no autorizado a datos de pago o credenciales es rechazado por el API Gateway y queda registrado para auditoría.	Alta
ASR-04	Respuesta bajo carga en apertura de venta	Rendimiento	Tiempo de respuesta	CU-001, CU-005	Durante la apertura de venta de un evento masivo, miles de usuarios consultan disponibilidad y compran de forma simultánea; la mayoría de las solicitudes se resuelve en tiempos cortos y estables.	Alta
ASR-05	Actualización de aforo y zonas en tiempo real	Tiempo real	Actualización de estado	CU-010, CU-019	El aforo y el estado de las zonas se reflejan en los tableros de monitoreo con baja latencia tras el registro de un ingreso o un incidente.	Media-Alta
ASR-06	Escalado independiente del servicio de Entradas	Escala-bilidad	Escala-bilidad horizontal	CU-001, CU-006	Ante un incremento súbito de tráfico en el servicio de Entradas durante una preventa, aumentan sus instancias sin afectar el rendimiento de los demás dominios.	Media

ID ASR	Título	Atributo	Sub-atributo	CU(s)	Escenario	Prioridad
ASR-07	Auditoría de transferencias y pagos	Trazabilidad	Audita-bilidad	CU-006, CU-031	Cada transferencia de entrada, pago o reembolso queda registrada con usuario, fecha y resultado, consultable en un reporte de auditoría.	Media
ASR-08	Interfaces especializadas por rol	Usabilidad	Adecuación a la tarea	Todos los CU (por interfaz)	Un usuario nuevo de cada rol completa su tarea principal en su interfaz correspondiente sin capacitación previa.	Baja-Media
ASR-09	Evolución independiente de dominios	Mantenibilidad	Modifica-bilidad	Todos los CU (por microservicio)	Un cambio en las reglas de negocio de Parqueaderos se implementa y despliega sin requerir cambios ni redespliegue del servicio de Entradas.	Baja-Media
ASR-10	Consumo uniforme vía API	Portabilidad	Independencia de plataforma	Todos los CU	Una misma funcionalidad de backend se consume igual desde el portal web y desde la app móvil, sin duplicar lógica de negocio.	Baja-Media

Tabla 5: Árbol de Utilidad — primera versión.

6. Restricciones

Restricción	Categoría	Descripción
Arquitectura de microservicios con base de datos independiente por dominio	Técnica	Decisión fundacional ya tomada por el equipo (ver ADR-01); cualquier solución a un ASR debe respetar esta separación por dominio.
Soporte de picos de miles de usuarios concurrentes	Técnica	El sistema debe soportar la concurrencia propia de la apertura de venta y el ingreso masivo al recinto sin degradar el servicio.
Equipo de 4 integrantes, 8 CU por persona	Organizacional	La distribución de trabajo (Daniel Cristancho, Samuel Emperador, Sebastián Sánchez, Diego Coronado) condiciona qué componentes prototipa cada integrante para Entrega 1 y Entrega 2.
Documento basado en el estándar arc42	Convencional	La plantilla y estructura de este documento las exige el curso; no pueden omitirse secciones.
Fechas y formato de entrega del curso	Convencional	Entrega 1 (SAD v1) y Entrega 2 (prototipo + SAD v2) deben respetar las fechas y reglas de entrega definidas por el curso.

7. Contexto y Alcance

El sistema interactúa con cinco roles de usuario (Cliente, Organizador, Empleado, Administrador y Supervisor de emergencia) y dos sistemas externos (Pasarela de Pagos y Proveedor de Notificaciones). La Figura 2 muestra el diagrama de contexto (Nivel 1 del modelo C4).

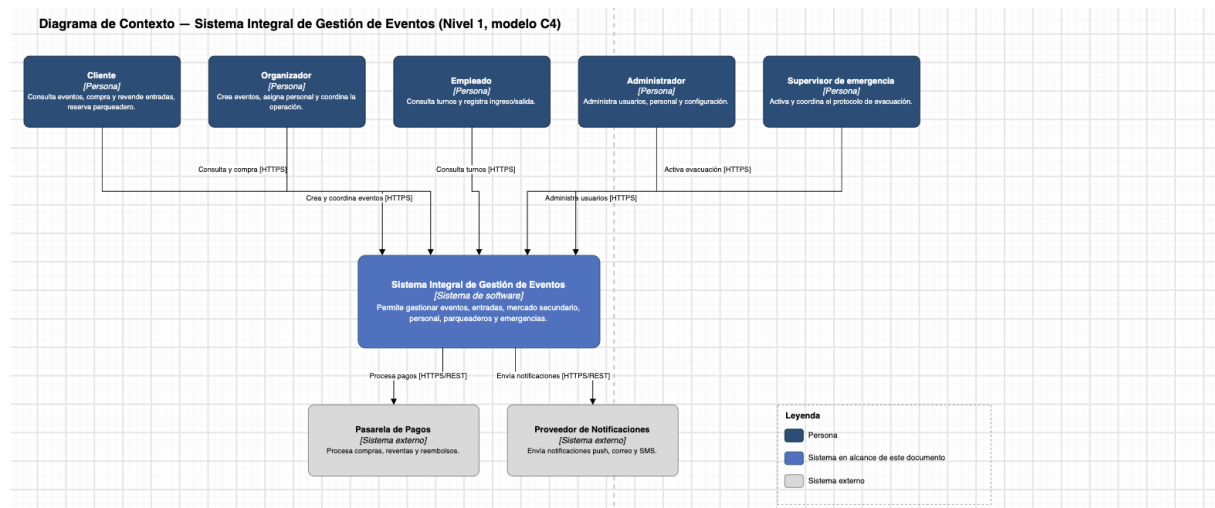


Figura 2: Diagrama de contexto (Nivel 1, modelo C4).

Origen	Destino	Descripción
Cliente	Sistema	Consulta eventos, compra y revende entradas, reserva parqueadero.
Organizador	Sistema	Crea eventos, asigna personal y coordina la operación.
Empleado	Sistema	Consulta turnos y registra ingreso/salida.
Administrador	Sistema	Administra usuarios, personal y configuración.
Supervisor de emergencia	Sistema	Activa y coordina el protocolo de evacuación.
Sistema	Pasarela de Pagos	Procesa compras, reventas y reembolsos.
Sistema	Proveedor de Notificaciones	Envía notificaciones push, correo y SMS.

Tabla 7: Relaciones del diagrama de contexto.

El diagrama de panorama de sistemas (*System Landscape, Diagrams/04-Panorama.png* en *C4Diagrams.tex*) amplía este alcance mostrando además cómo el sistema se relaciona con otros sistemas de la organización (Contabilidad y Facturación, CRM/Marketing y una plataforma de Analítica), fuera del alcance de este documento pero relevantes para entender el ecosistema completo.

8. Vista de contenedores

El sistema se divide en cuatro interfaces desplegables (Portal Web del Cliente, Portal Web Administrativo/Operativo, App Móvil del Cliente y App Móvil del Personal), todas consumiendo el mismo backend a través de un API Gateway. El backend está compuesto por microservicios independientes por dominio (Entradas/Mercado Secundario, Personal, Eventos/Emergencias, Parqueaderos) y por infraestructura de soporte (Redis, cola de mensajes y bases de datos por servicio). La Figura 3 presenta el diagrama de contenedores.

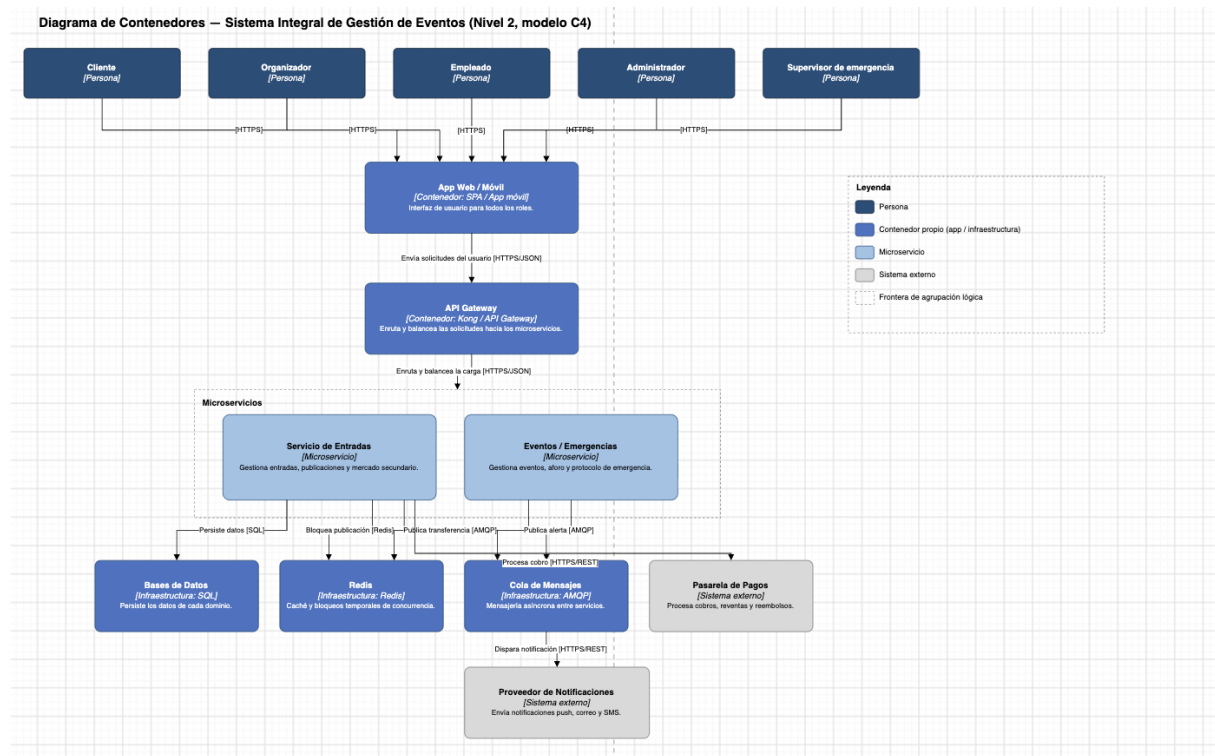


Figura 3: Diagrama de contenedores (Nivel 2, modelo C4).

- **API Gateway + balanceador de carga:** punto único de entrada al backend; centraliza autenticación, autorización, validación de tokens y enrutamiento hacia el microservicio correspondiente.
- **Microservicios:** Entradas/Mercado Secundario, Personal, Eventos/Emergencias y Parqueaderos, cada uno con su propia base de datos para reducir el acoplamiento entre dominios.
- **Redis:** bloqueo temporal de recursos en disputa (p. ej. una entrada en reventa durante el checkout) y datos de acceso rápido (aforo, ocupación, sesiones).
- **Cola de mensajes (RabbitMQ/Kafka):** desacopla operaciones asíncronas: generación de QR, notificaciones, auditoría, liquidación de pagos y alertas de emergencia.
- **Servicios externos:** Pasarela de Pagos y Proveedor de Notificaciones, integrados vía API

y, en el caso de notificaciones, disparados de forma asíncrona desde la cola de mensajes.

9. Vista de componentes

Se detalla primero el contenedor **Servicio de Entradas y Mercado Secundario**, por ser el que resuelve el caso de uso complejo CU-006 (Gestión del Mercado Secundario de Entradas), y luego, al mismo nivel de detalle, los otros tres microservicios de dominio. La Figura 4 muestra los componentes internos del Servicio de Entradas.

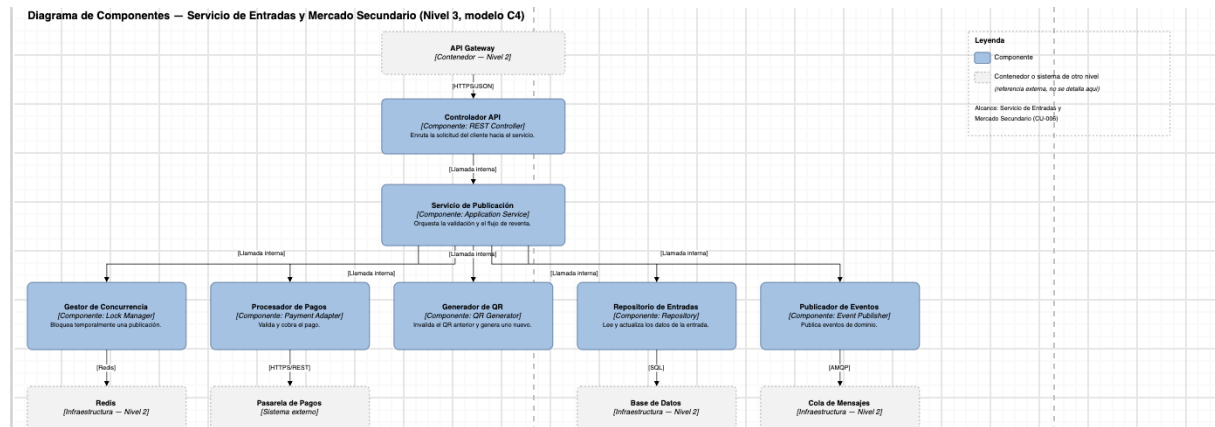


Figura 4: Diagrama de componentes — Servicio de Entradas y Mercado Secundario (Nivel 3, modelo C4).

El flujo interno es: el **Controlador API** recibe la solicitud enrutada por el API Gateway y la delega al **Servicio de Publicación**, que orquesta la operación de reventa. Este componente solicita al **Gestor de Concurrencia** el bloqueo temporal de la entrada en Redis, invoca al **Procesador de Pagos** para cobrar la transacción contra la Pasarela de Pagos, usa el **Generador de QR** para invalidar el código anterior y emitir uno nuevo, persiste el cambio de propiedad mediante el **Repositorio de Entradas**, y finalmente notifica el resultado a través del **Publicador de Eventos** hacia la cola de mensajes.

9.1. Servicio de Personal

El **Controlador API** enruta la solicitud hacia el **Servicio de Asignación de Personal**, que valida rol, disponibilidad y horario del empleado con apoyo del **Validador de Conflictos** antes de confirmar una asignación o un cambio de turno. El **Gestor de Turnos** calcula horarios y, junto con el **Repositorio de Personal**, persiste asignaciones, turnos y registros de asistencia. El **Publicador de Eventos** notifica, vía cola de mensajes, la confirmación de asignaciones y cambios de turno al empleado y al supervisor.

9.2. Servicio de Eventos / Emergencias

El **Controlador API** enruta hacia el **Servicio de Gestión de Eventos** (CRUD de eventos, localidades y aforo) o hacia el **Servicio de Protocolo de Emergencia**, según la operación.

Al activarse una evacuación, este último consulta al **Gestor de Zonas y Aforo** (apoyado en Redis para datos de baja latencia) para identificar zonas afectadas, salidas y rutas disponibles; persiste el incidente mediante el **Repositorio de Eventos**; y usa el **Publicador de Eventos** para difundir alertas y actualizaciones de estado a través de la cola de mensajes hacia el personal y los asistentes.

9.3. Servicio de Parqueaderos

El **Controlador API** delega en el **Servicio de Reservas**, que gestiona la reserva previa y el ingreso/salida de vehículos. El **Gestor de Ocupación** mantiene en Redis el estado de ocupación por zona en tiempo real. El **Procesador de Cobro** invoca la Pasarela de Pagos para tarifas y reservas. El **Repositorio de Espacios** persiste las asignaciones, y el **Publicador de Eventos** notifica confirmaciones hacia la cola de mensajes.

10. Vista de procesos

Se documentan los dos procesos más críticos del sistema desde el punto de vista de los atributos de calidad: la reventa de una entrada (conurrencia y consistencia) y la evacuación ante una emergencia (disponibilidad y tiempo real).

10.1. Reventa de una entrada (CU-006)

La Figura 5 muestra, con líneas de vida y mensajes numerados, cómo colaboran los contenedores para resolver una compra en el mercado secundario.

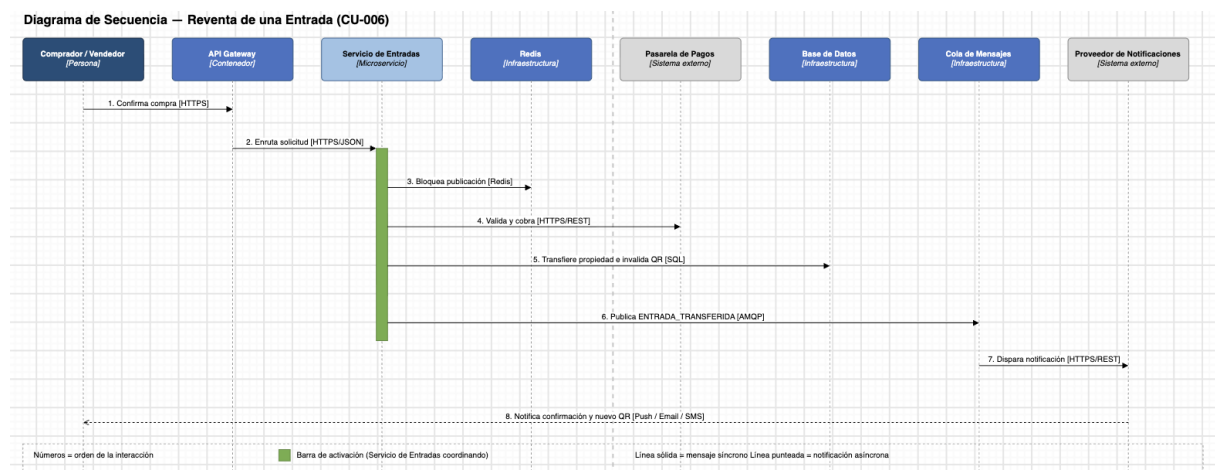


Figura 5: Diagrama de secuencia — reventa de una entrada (CU-006).

1. El comprador confirma la compra de una entrada publicada en el mercado secundario.
2. El API Gateway enruta la solicitud autenticada hacia el Servicio de Entradas.
3. El Servicio de Entradas bloquea temporalmente la publicación en Redis, para impedir que otro comprador la adquiera de forma simultánea.

4. Se valida el medio de pago y se cobra el valor de la entrada mediante la Pasarela de Pagos.
5. Se transfiere la propiedad de la entrada y se invalida el código QR anterior en la base de datos.
6. Se publica el evento ENTRADA_TRANSFERIDA en la cola de mensajes.
7. El Proveedor de Notificaciones envía la confirmación y el nuevo código QR al comprador y al vendedor, de forma desacoplada del flujo principal.

10.2. Evacuación ante emergencias (CU-010)

1. El supervisor activa el protocolo de evacuación desde el Portal Web Administrativo, registrando el tipo y ubicación de la emergencia.
2. El servicio de Eventos/Emergencias identifica las zonas potencialmente afectadas y consulta el aforo actual, las salidas disponibles y las rutas configuradas.
3. El sistema presenta las rutas y zonas recomendadas; el supervisor confirma el protocolo.
4. Se notifica al personal operativo y se envían instrucciones a los asistentes mediante la cola de mensajes y el Proveedor de Notificaciones.
5. El sistema monitorea las zonas en tiempo real mientras el supervisor atiende alertas (ruta bloqueada, zona congestionada, cambio de nivel de emergencia) hasta confirmar la finalización.

11. Vista física

La Figura 6 muestra el ambiente de **producción**: los dispositivos del usuario acceden vía CDN a los archivos estáticos y vía balanceador + API Gateway al backend, que corre en un clúster de aplicación con réplicas por microservicio, respaldado por un clúster de bases de datos, un clúster Redis y un clúster de mensajería.

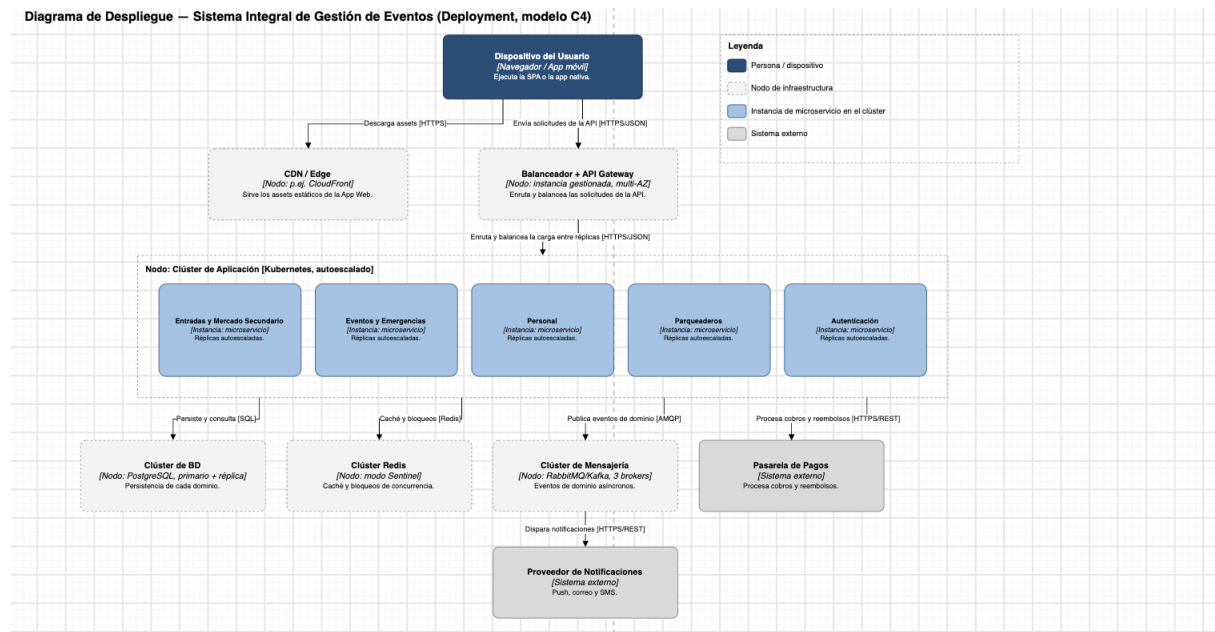


Figura 6: Diagrama de despliegue — ambiente de producción (modelo C4).

Los ambientes de **desarrollo** y **pruebas** reutilizan la misma topología del diagrama de despliegue de producción, a menor escala: una única réplica por microservicio (sin balanceo horizontal), sin CDN (los archivos estáticos se sirven directamente desde el servidor de aplicación), y con Redis, la cola de mensajes y las bases de datos desplegadas en un único nodo compartido en vez de un clúster. El ambiente de **desarrollo** se despliega automáticamente mediante integración continua ante cada cambio en la rama principal, con datos de prueba mínimos para verificación funcional rápida. El ambiente de **pruebas** se usa para validación funcional end-to-end y pruebas de carga previas a cada entrega, con datos sintéticos que simulan la concurrencia esperada durante la apertura de venta y el ingreso masivo, de forma que los ASR de disponibilidad y rendimiento (sección 5) puedan verificarse antes de producción. El script de despliegue automatizado que provisiona ambos ambientes se entregará junto con el prototipo funcional en la Entrega 2.

12. Modelo de datos

Siguiendo la decisión de una base de datos independiente por microservicio (ADR-01), cada dominio tiene su propio esquema relacional. Las referencias entre dominios (p. ej. el propietario de una *Entrada* hacia *Usuario*, que vive en el módulo de administración de cuentas) se resuelven a nivel de aplicación mediante el identificador correspondiente, y no con llaves foráneas físicas entre bases de datos distintas, lo cual es consistente con la separación de datos establecida en ADR-01. A continuación se listan las entidades principales de cada esquema, con sus llaves foráneas lógicas.

Esquema de Entradas / Mercado Secundario

- **Entrada**(id, evento_id, localidad_id, propietario_id, codigoQR, estado)

- **PublicacionReventa**(id, entrada_id, vendedor_id, precio, estado)

Esquema de Personal

- **Personal**(id, usuario_id, rolOperativo)
- **Turno**(id, personal_id, evento_id, zona_id, horario)
- **RegistroAsistencia**(id, turno_id, horaEntrada, horaSalida)

Esquema de Eventos / Emergencias

- **Evento**(id, nombre, fecha, recinto_id, aforo)
- **Recinto**(id, nombre, direccion)
- **Zona**(id, recinto_id, capacidad)
- **Incidente**(id, evento_id, tipo, estado)

Esquema de Parqueaderos

- **Parqueadero**(id, evento_id, capacidad)
- **EspacioParqueadero**(id, parqueadero_id, estado)

Esquemas de administración general

- **Usuario**(id, nombre, email, credenciales), **Rol**(id, nombre), **UsuarioRol**(usuario_id, rol_id)
- **Proveedor**(id, nombre, servicio)
- **Pago**(id, tipo, monto, estado, referencia_id, referencia_tipo) — registra pagos de Entradas, Pedidos o Parqueaderos mediante **referencia_tipo**
- **Pedido**(id, cliente_id, establecimiento_id, estado, total), **Establecimiento**(id, nombre), **Producto**(id, establecimiento_id, nombre, precio)
- **Reporte**(id, tipo, fechaGeneracion)

El motor de base de datos concreto por dominio es **PostgreSQL** para los esquemas transaccionales y **MongoDB** para Reportes (ADR-06).

13. Registros de Decisiones Arquitectónicas (ADR)

ADR-01 — Arquitectura de microservicios por dominio

ASR relacionados: ASR-06, ASR-09.

Problema: el sistema debe permitir que sus dominios (Entradas, Mercado Secundario, Personal, Eventos, Parqueaderos, Emergencias) evolucionen y escalen de forma relativamente independiente, sin que un cambio en uno arriesgue la disponibilidad de los demás.

Solución: se adopta una arquitectura de **microservicios**, uno por dominio, cada uno con base de datos propia.

Alternativa descartada: monolito modular — dificulta escalar de forma independiente el servicio de Entradas durante una apertura de venta sin sobre-aprovisionar los demás dominios.

Consecuencias: favorece mantenibilidad y escalabilidad independiente, a costa de mayor complejidad operativa (más piezas de infraestructura) y de tener que gestionar consistencia eventual entre servicios en vez de transacciones ACID únicas.

ADR-02 — API Gateway y balanceador de carga

ASR relacionados: ASR-02, ASR-03.

Problema: se necesita un punto único, seguro y disponible de entrada al backend para las cuatro interfaces del sistema.

Solución: un **API Gateway** centraliza autenticación, autorización y enrutamiento; un **balanceador de carga** distribuye solicitudes entre múltiples instancias de cada microservicio.

Alternativa descartada: exponer cada microservicio directamente a los clientes — se descartó por duplicar lógica de seguridad en cada interfaz y complicar el enrutamiento.

Consecuencias: centraliza seguridad y facilita el enrutamiento, pero introduce un componente crítico que debe replicarse (mitigado con múltiples instancias detrás del balanceador) para no convertirse en un punto único de falla.

ADR-03 — Redis para bloqueo temporal y datos de acceso rápido

ASR relacionados: ASR-01, ASR-05.

Problema: evitar que dos compradores completen simultáneamente la compra de la misma entrada en reventa, y disponer de datos de aforo/ocupación con baja latencia.

Solución: **Redis** se usa como bloqueo distribuido temporal durante el checkout de una reventa, y como caché de datos de acceso rápido (aforo, ocupación, sesiones).

Alternativa descartada: bloqueo pesimista directamente en la base de datos relacional — se descartó por convertirse en cuello de botella bajo la carga concurrente esperada.

Consecuencias: resuelve la doble venta y acelera consultas frecuentes, pero añade un punto adicional de falla que requiere su propia estrategia de disponibilidad (réplica/clúster).

ADR-04 — Cola de mensajes para operaciones asíncronas

ASR relacionados: ASR-04, ASR-07.

Problema: operaciones como notificaciones o liquidación de pagos no deben bloquear ni hacer fallar la operación principal (compra, transferencia) cuando un servicio externo es lento o falla temporalmente.

Solución: una **cola de mensajes (RabbitMQ/Kafka)** desacopla la generación de QR, las notificaciones, la auditoría y la liquidación de pagos del flujo síncrono principal.

Alternativa descartada: comunicación síncrona directa con los servicios externos — se descartó porque un fallo temporal del proveedor de notificaciones no debe bloquear la operación principal.

Consecuencias: tolera fallos externos y mejora el rendimiento percibido, a costa de consistencia

eventual y de requerir monitoreo propio de las colas.

ADR-05 — Stack de frontend por tipo de interfaz

ASR relacionados: ASR-08, ASR-10, ASR-05.

Problema: las cuatro interfaces (portal web del cliente, portal web administrativo/operativo, app móvil del cliente, app móvil de personal) deben implementarse en un lenguaje/framework concreto, sin comprometer el consumo uniforme del backend vía API Gateway (ASR-10) ni la especialización por rol de cada interfaz (ASR-08).

Solución: los portales web se construyen en **Angular + TypeScript** — su estructura de módulos, *guards* e *interceptors* favorece el trabajo en paralelo de varios integrantes con convenciones consistentes, y RxJS encaja con las actualizaciones en tiempo real de ASR-05. Las apps móviles se construyen en **Kotlin nativo sobre Android Studio** (Jetpack Compose para la interfaz, *coroutines/Flow* para tiempo real), aprovechando que el equipo ya domina ambos lenguajes.

Alternativa descartada: un framework híbrido/*cross-platform* (p. ej. React Native o Flutter) para compartir código de interfaz entre web y móvil — se descartó porque ASR-10 ya evita duplicar lógica de negocio en el cliente (esta vive en el backend, detrás del API Gateway), por lo que compartir código de UI entre plataformas no aporta un beneficio arquitectónico claro frente al costo de una capa de abstracción adicional y a la experiencia ya existente del equipo con Angular y Kotlin nativos.

Consecuencias: cada interfaz evoluciona con las herramientas nativas de su plataforma sin las limitaciones de un framework híbrido, a costa de mantener dos ecosistemas de *build/* pruebas distintos (TypeScript/npm y Kotlin/Gradle) y de no compartir código de UI entre web y móvil — los contratos comunes quedan limitados a la especificación de la API (ver `App/shared/`).

ADR-06 — Motor de base de datos por dominio (*polyglot persistence*)

ASR relacionados: ASR-01, ASR-07.

Problema: ADR-01 estableció una base de datos propia por microservicio, pero dejó abierto el motor concreto; los esquemas transaccionales (Entradas, Personal, Eventos/Emergencias, Parqueaderos, cuentas/proveedores/pagos) tienen necesidades de consistencia distintas a las de Reportes/analítica, cuyos datos son más variables y semi-estructurados.

Solución: **PostgreSQL** como motor relacional para todos los esquemas transaccionales (uno por microservicio, ADR-01), y **MongoDB** como motor documental exclusivamente para el esquema de Reportes dentro del servicio de administración.

Alternativa descartada: un único motor relacional (PostgreSQL) también para Reportes — se descartó para no forzar un esquema rígido sobre datos de reportes cuya estructura varía entre tipos de reporte, tal como ya anticipaba la sección de modelo de datos.

Consecuencias: cada esquema usa el motor más adecuado a su naturaleza (ACID para lo transaccional, esquema flexible para reportes), a costa de operar y respaldar dos motores de base de datos distintos en vez de uno solo.

14. Riesgos técnicos

Riesgo	Probabili- dad	Impacto	Priori- dad	Plan de miti- gación	Plan de con- tingencia
Concurrencia entre compradores en el mercado secundario	Media	Alto	Alta	Bloqueo distribuido con TTL en Redis antes de procesar el pago.	Rechazar la segunda transacción y notificar de inmediato al comprador no atendido.
Caída del API Gateway como punto único de entrada	Baja	Alto	Alta	Múltiples instancias + balanceador de carga + <i>health checks</i> .	Failover automático hacia una instancia o región de respaldo.
Indisponibilidad de la pasarela de pagos externa	Media	Alto	Alta	<i>Timeouts</i> , reintentos y cola de reconciliación de pagos.	Marcar la transacción como pendiente y permitir reintento manual del usuario.
Picos de tráfico en apertura de venta / ingreso masivo	Alta	Medio	Alta	Autoescalado del servicio de Entradas + caché en Redis.	Sala de espera virtual (cola de acceso) para limitar la concurrencia admitida.
Fallo del proveedor de notificaciones durante una emergencia	Baja	Alto	Media	Reintentos automáticos + canal secundario (SMS si falla push).	Escalar el aviso de forma manual mediante el personal en sitio.
Pérdida de conexión de una app móvil durante una evacuación	Media	Medio	Media	Caché local de instrucciones + reconexión automática.	El personal de seguridad releva la comunicación directamente en sitio.

15. Glosario

- **SAD:** Descripción de la Arquitectura del Software (*Software Architecture Description*), este documento.
- **ASR:** Requisito Arquitectónicamente Significativo (*Architecturally Significant Require-*

ment).

- **ADR:** Registro de Decisión Arquitectónica (*Architecture Decision Record*).
- **CU:** Caso de Uso.
- **QA:** Atributo de Calidad (*Quality Attribute*).
- **Modelo C4:** notación de Simon Brown para diagramar arquitectura en niveles (Contexto, Contenedores, Componentes, Código).
- **API Gateway:** componente que centraliza el enrutamiento, la autenticación y la autorización de las solicitudes al backend.
- **Microservicio:** servicio backend independiente, responsable de un dominio de negocio, con su propia base de datos.
- **Redis:** base de datos en memoria usada como caché y para bloqueos distribuidos.
- **RabbitMQ / Kafka:** sistemas de colas de mensajes usados para procesamiento asíncrono entre servicios.
- **AMQP:** protocolo de mensajería usado por colas como RabbitMQ.
- **QR:** código de respuesta rápida (*Quick Response*) usado para identificar entradas y validar pedidos.
- **Aforo:** cantidad de personas presentes o permitidas en un recinto o zona.
- **Mercado secundario:** reventa de entradas ya adquiridas entre usuarios del sistema.
- **CDN:** red de distribución de contenido (*Content Delivery Network*) usada para servir archivos estáticos cerca del usuario.
- **Balanceador de carga:** componente que distribuye solicitudes entre varias instancias de un servicio.
- **Bloqueo distribuido:** mecanismo (implementado aquí con Redis) que impide que dos procesos modifiquen simultáneamente el mismo recurso.
- **Consistencia eventual:** garantía de que los datos de distintos servicios convergen al mismo estado, aunque no de forma instantánea.

16. Referencias

- arc42. *arc42 overview*. <https://arc42.org/overview>
- Brown, S. *The C4 model for visualising software architecture*. <https://c4model.com>
- Brown, S. *Software architecture diagram review checklist*. <https://c4model.com/diagrams/checklist>

- Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley. (Base teórica del Árbol de Utilidad y los ADR usados en este documento.)
- Documentos internos del proyecto: *ArchitecturalProposal.tex*, *C4Diagrams.tex*, *Submission/DistribucionCasosUso.pdf* y *BitacoraArquitectonica.md* (equipo Hexacore, 2026).